

# 機械学習 Classification

最も単純な 2 クラス分類

rkskmt

1

## 分類問題

### 目標

- 分類問題(Data Classification)：データを分類するタスクを理解する  
→ 2クラス分類
- データ可視化
- **決定境界 (decision boundary)** の概念を学ぶ

### ① ここがポイント

- 完全な分類は困難であることを知る
- 境界決定のトレードオフとコスト

2

## 今回の問題

### サケだけを選別せよ

- 魚はベルトコンベアで大量に流れてくる  
→ 時間をかけず選別したい
- サケ以外の魚も混じっている  
→ 一旦、**サケ** (Salmon) か **スズキ** (Sea bass) か  
→ 分類項目を **クラス (class)** という

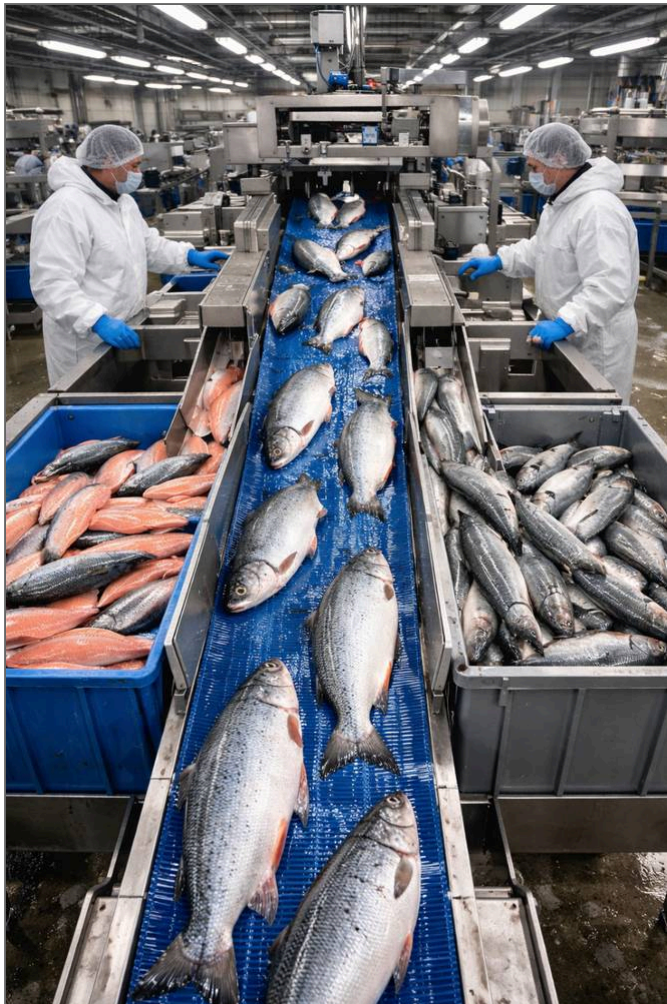
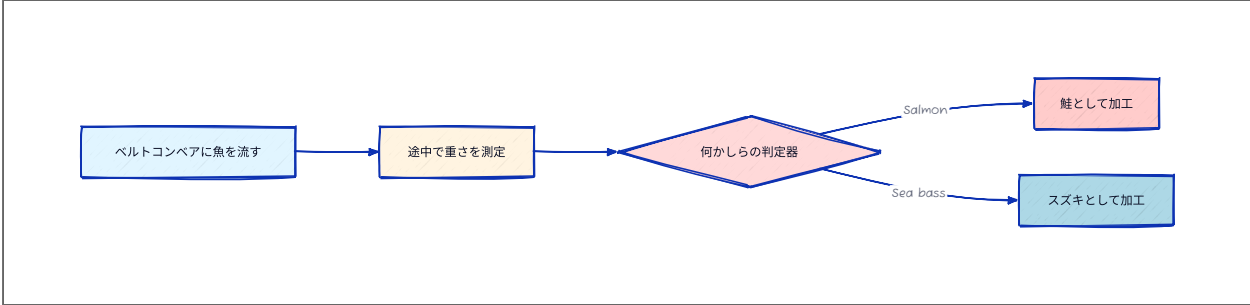


### ① ここがポイント

Salmon と Sea bass の **2クラス識別(2-class classification)** 問題

3

# 選別システム



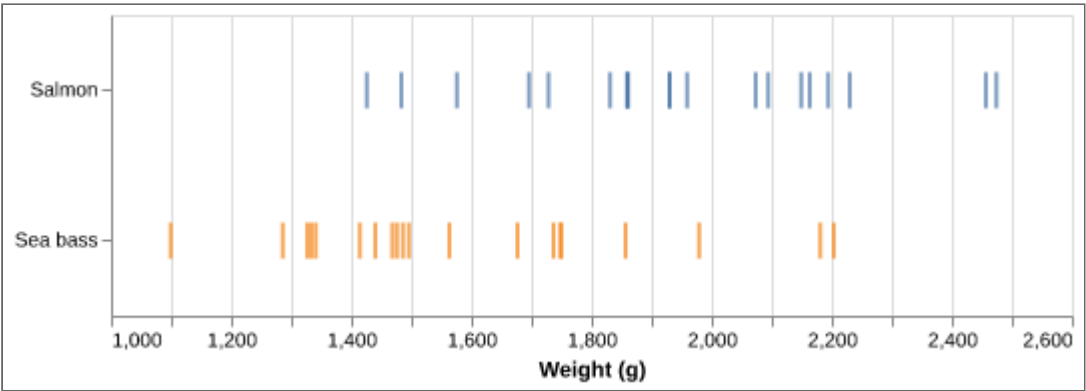
## 試行1: 重さだけで分類してみる

### アプローチ

- 重量計をつけられそうなので、まず重さで識別できないかを考えた
- まずサンプルとして20匹ずつの重さを測定した

```
1 # Salmon (20匹) の重さ (g)
2 salmon_weights = [2149, 1959, 2194, 2457, 1930, 1930, 2474, 2230, 1859, 2163,
3                   1861, 1860, 2073, 1426, 1483, 1831, 1696, 2094, 1728, 1576]
4
5
6 # Sea bass (20匹) の重さ (g)
7 seabass_weights = [1677, 1327, 1486, 1440, 1857, 1476, 1749, 2203, 1979, 1468,
8                   1496, 1737, 1099, 1341, 1748, 1563, 2181, 1414, 1286, 1333]
```

Python



## 準備

pandas, numpy, matplotlib, scikit-learnをインストールします。

```
1 pip install pandas numpy matplotlib scikit-learn altair[all]
```

Google Colabの場合は、altair[all]だけインストールすればOKです

```
1 !pip install altair[all]
```

Python

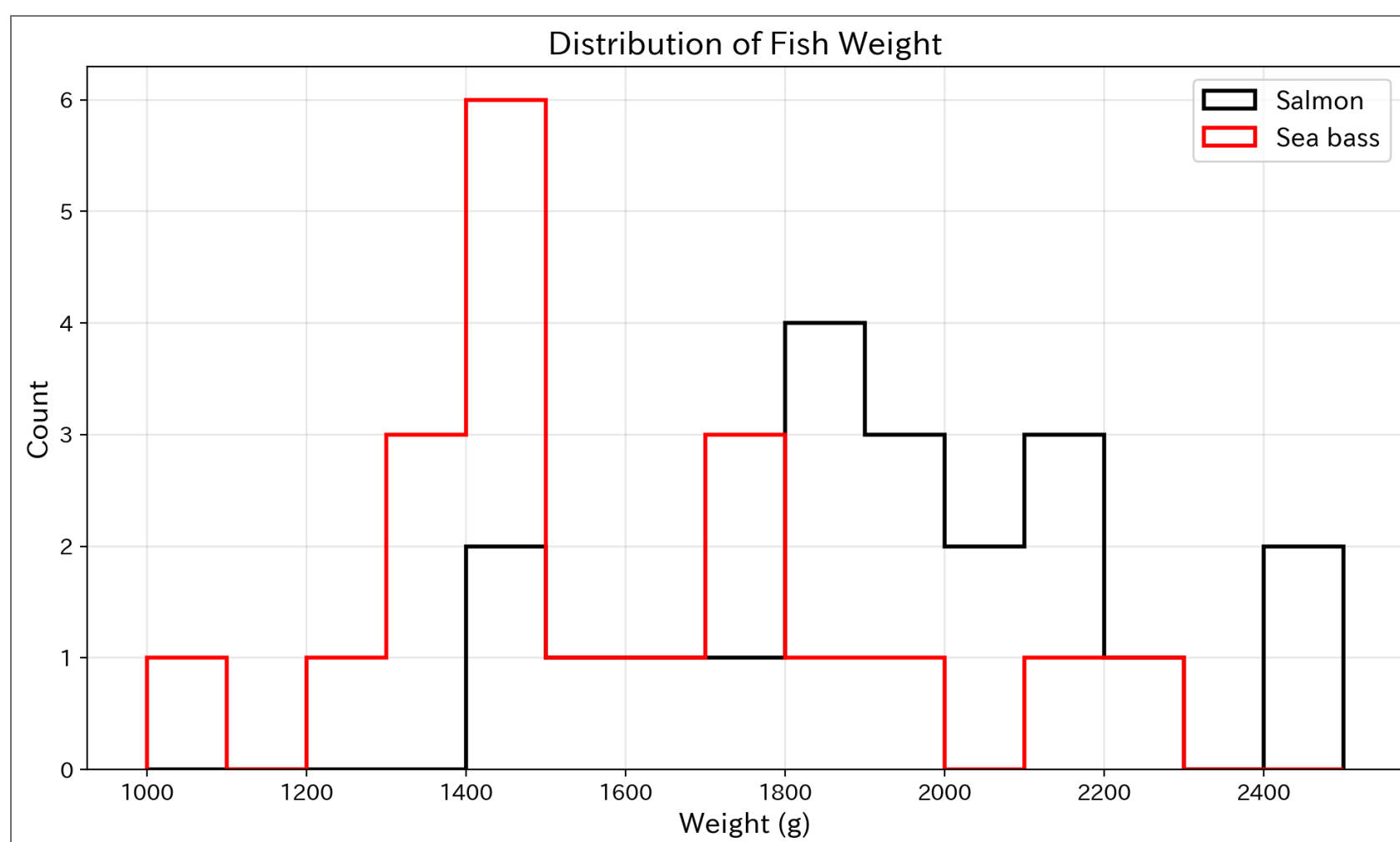
以下を実行

▶ 全体を表示

6

## まずはヒストグラム

▶ code



7

## 統計値を見る

▶ code

Salmon - 最小: 1426g, 最大: 2474g, 平均: 1949g  
Sea bass - 最小: 1099g, 最大: 2203g, 平均: 1593g

重なり範囲: 1426g - 2203g

### ⚠ 発見

- **双峰型の分布** : 2つの山が見える  
→ **重なっている** がまあ分けられそう？

8



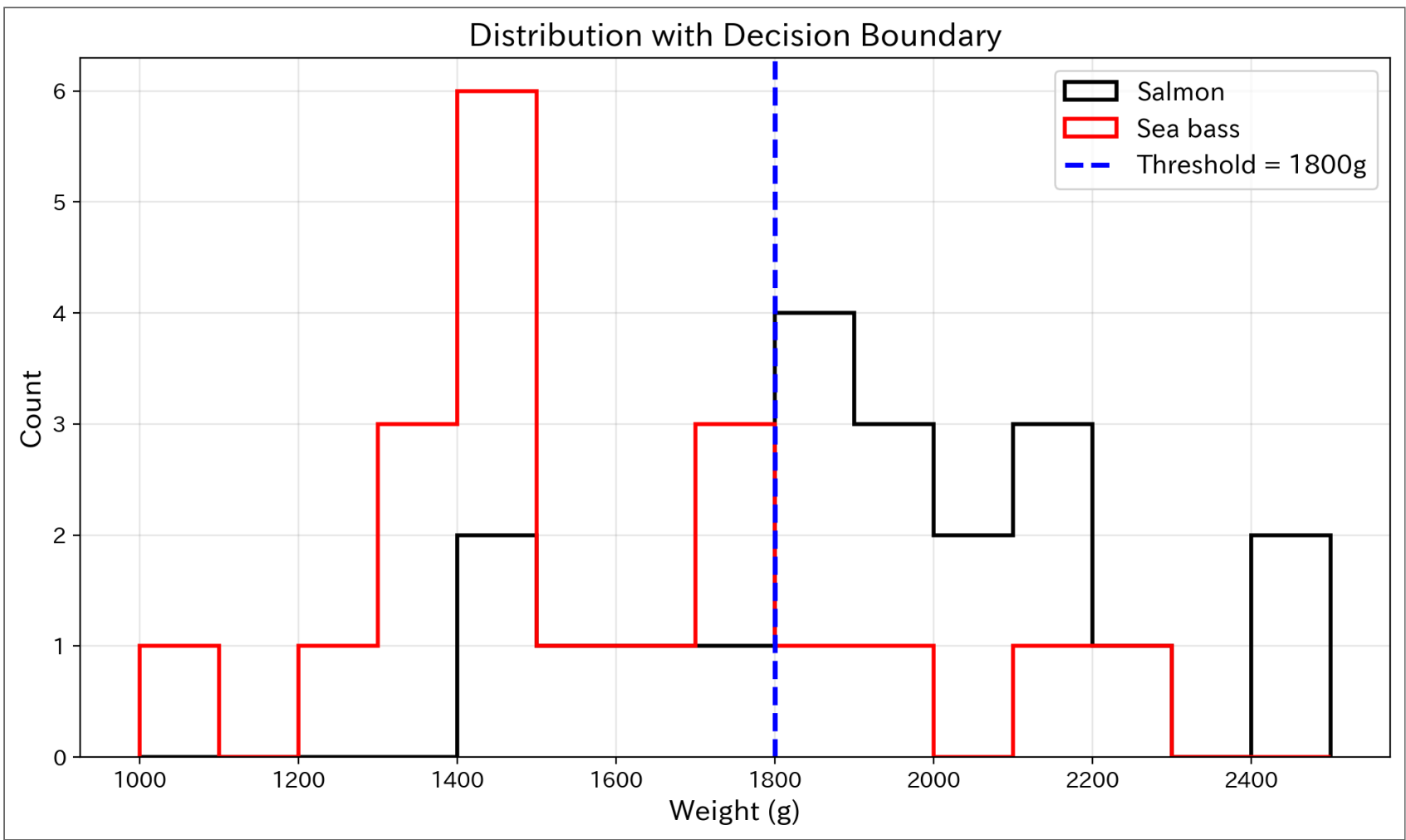
境界線（閾値）を考える

大体でいいので **境界線（threshold）** を設定します。

方針

- 1800g以上** → Salmon
- 1800g未満** → Sea bass
- この判定ルールで分類してみましょう。

青破線で切ったとしたら



分類結果：4つのパターン

境界線1800gで分類すると、4つのパターンが生まれます：

| 実際の魚     | 判定結果     | 略称                  | 説明                  |
|----------|----------|---------------------|---------------------|
| Salmon   | Salmon   | TP (True Positive)  | 正しく Salmon と判定      |
| Sea bass | Sea bass | TN (True Negative)  | 正しく Salmon ではない と判定 |
| Sea bass | Salmon   | FP (False Positive) | 誤って Salmon と判定      |
| Salmon   | Sea bass | FN (False Negative) | Salmon を 見逃し        |

# TP/FP/TN/FNの覚え方

## 右から左に読む

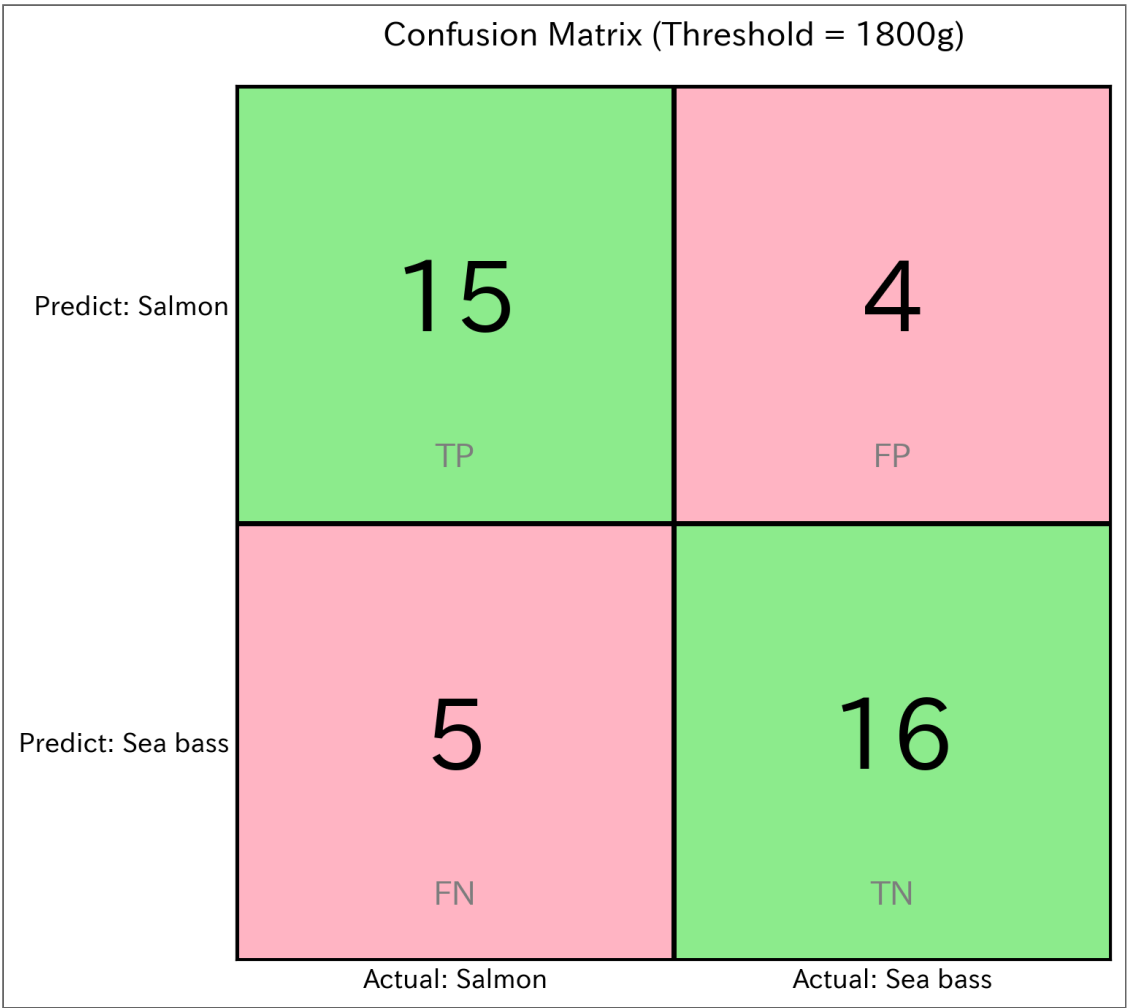
- 1. 右 (P/N) 判定結果 → Positive (シャケ) or Negative (それ以外)
- 2. 左 (T/F) その答え合わせ\*\* → True (正解) or False (不正解)

つまり、ある魚をみて識別した結果、

- **TP**: Positiveと判定 → True (正しかった！)
- **FP**: Positiveと判定 → False (間違ってた...)
- **TN**: Negatigeと判定 → True (正しかった！)
- **FN**: Negativeと判定 → False (間違ってた...)

# 混同行列 (Confusion Matrix)

▶ code



# 判定器の評価

▶ code

| 指標                                                                     | 値     | 意味                        |
|------------------------------------------------------------------------|-------|---------------------------|
| Accuracy                                                               | 0.775 | 全体の正解率                    |
| Precision                                                              | 0.789 | Salmonと判定した中で実際にSalmonの割合 |
| Recall                                                                 | 0.750 | 実際のSalmonをどれだけ検出できたか      |
| Accuracy = $\frac{TP + TN}{全体}$ = $\frac{正しく分類できた数}{全データ数}$            |       |                           |
| Precision = $\frac{TP}{TP + FP}$ = $\frac{正しいSalmon判定}{Salmonと判定した全て}$ |       |                           |
| Recall = $\frac{TP}{TP + FN}$ = $\frac{検出できたSalmon}{実際のSalmon全て}$      |       |                           |



# しきい値を変えると？

▶ code

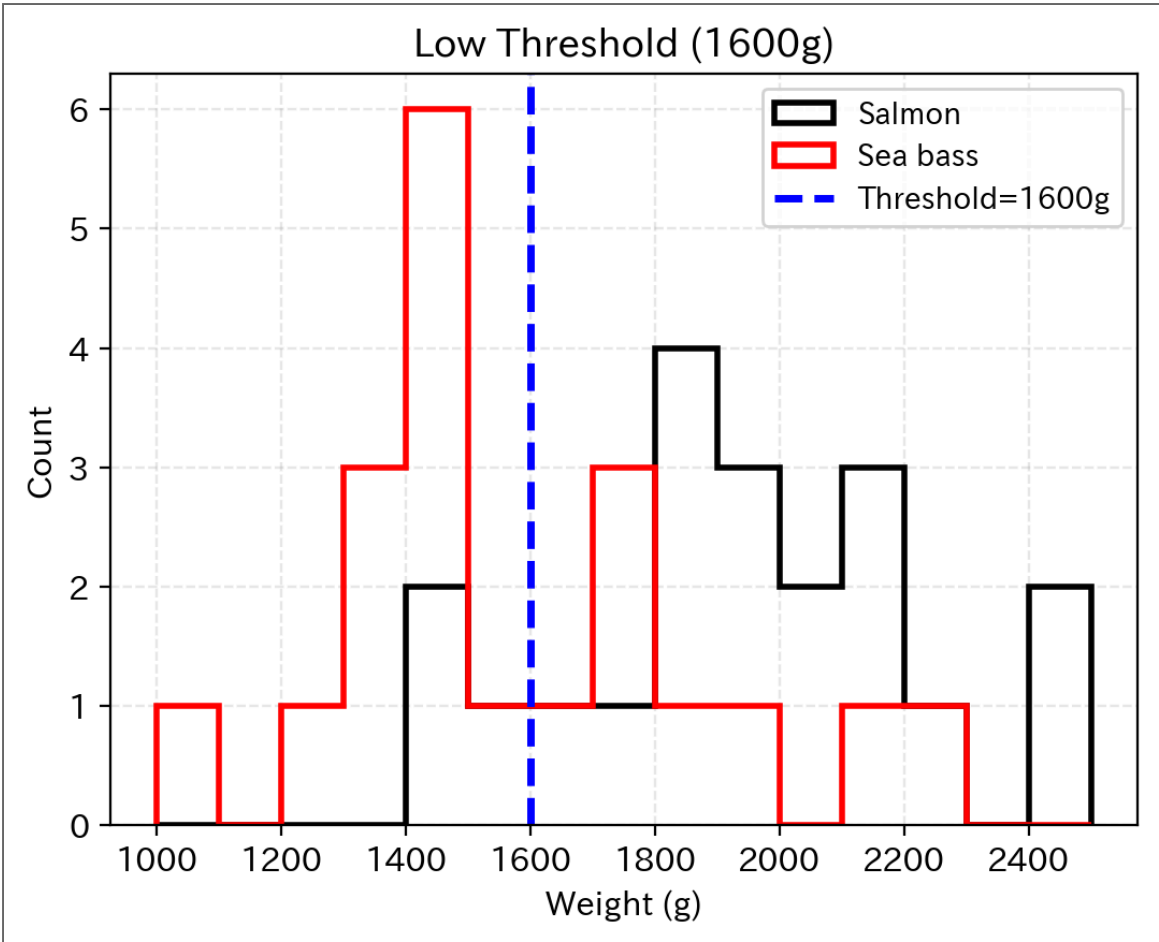
|   | 境界線   | TP | FP | FN | TN | Precision | Recall | Accuracy |
|---|-------|----|----|----|----|-----------|--------|----------|
| 0 | 1600g | 17 | 8  | 3  | 12 | 68.0%     | 85.0%  | 72.5%    |
| 1 | 1800g | 15 | 4  | 5  | 16 | 78.9%     | 75.0%  | 77.5%    |
| 2 | 2000g | 8  | 2  | 12 | 18 | 80.0%     | 40.0%  | 65.0%    |

## ❗ 重要

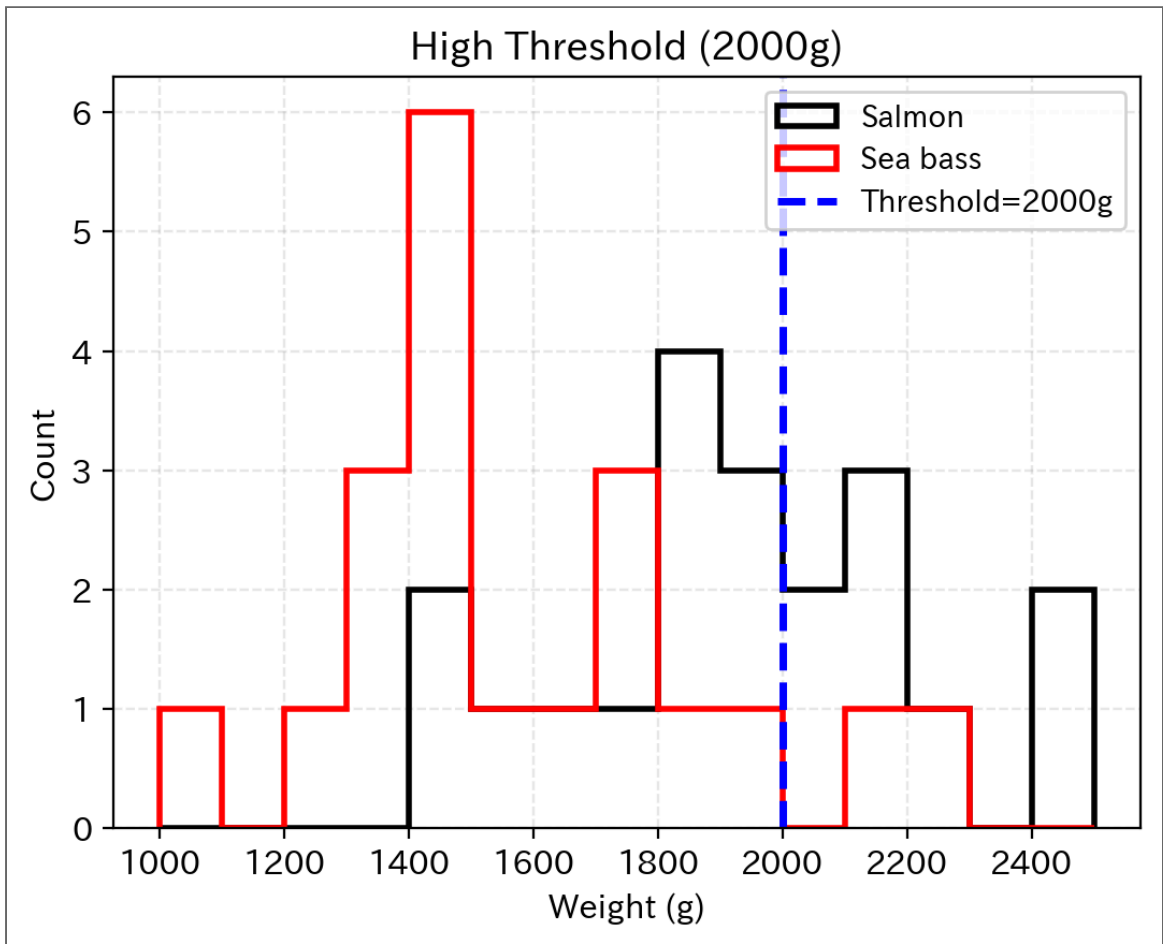
- **境界を下げる（1600g）** → Recall ↑、Precision ↓  
→ Salmonを逃さない（FN減）が、Sea bassの誤判定（FP増）
- **境界を上げる（2000g）** → Precision ↑、Recall ↓  
→ Salmon判定の信頼性は高いが、Salmonを見逃す（FN増）

**Precision と Recall はトレードオフの関係**

# しきい値の比較（1600g vs 2000g）



- FP=8
- FN=3
- **Precision**: 68.0% (低い)
- **Recall**: 85.0% (高い)



FP=2, FN=12 **Precision**: 80.0% (高い)  
**Recall**: 40.0% (低い)

# どの指標を重視すべきか？

## ケース1: 高級レストラン向け出荷

- **Precision重視** (FPを避ける)
- Sea bassの混入を最小化。混入絶対許さないマン
- → 高めの閾値 (2000g)

## ケース2: 缶詰工場 (Salmon専用ライン)

- **Recall重視** (FNを避ける)
- Salmonの取りこぼしを最小化。シャケ全部入れるマン
- → 低めの閾値 (1600g)

17

# コストを考慮した最適化

誤分類のコストが異なる場合、総コストを最小化する閾値を選びます。

$$\operatorname{argmin}(\text{総コスト}) = N(\text{FP}) \times \text{Cost}(\text{FP}) + N(\text{FN}) \times \text{Cost}(\text{FN})$$

$N(x)$  :  $x$ の個数,  $\text{Cost}(x)$  :  $x$  1個あたりのコスト

▶ code

| 境界線            | FP | FN | 総コスト    |
|----------------|----|----|---------|
| <b>1600g</b>   | 8  | 3  | ¥81,500 |
| <b>1800g</b>   | 4  | 5  | ¥42,500 |
| <b>★ 2000g</b> | 2  | 12 | ¥26,000 |

このコスト関数での最適なしきい値: 2000g (総コスト最小)

18

# 試行1の結論

## ❗ 重要なポイント

1. **重さだけでは完全な分類は不可能**  
→ データの分布が重なっている
2. **境界線は「方針の選択」**  
→ 正解/不正解ではなく、用途に応じた最適化 (optimization)
3. **評価指標はトレードオフ**  
→ Precision ↑ → Recall ↓  
→ どちらを重視するかは用途次第

19



# 練習問題

下記のデータ（それぞれ20匹ずつ追加した）について、以下の0-3までのことを実行して分類器をつくれ。

```
1 salmon_weights = [2149, 1959, 2194, 2457, 1930, 1930, 2474, 2230, 1859, 2163,  
2                     1861, 1860, 2073, 1426, 1483, 1831, 1696, 2094, 1728, 1576,  
3                     2028, 1903, 2057, 1629, 1884, 1884, 2240, 2081, 1838, 2037,  
4                     1839, 1738, 1978, 1754, 1991, 1720, 1731, 1992, 1752, 1652]  
5 seabass_weights = [1677, 1327, 1486, 1440, 1857, 1476, 1749, 2203, 1979, 1468,  
6                     2015, 1528, 1612, 1183, 1436, 1625, 1261, 1701, 1420, 1509,  
7                     1387, 1345, 1505, 1204, 1381, 1513, 1259, 1567, 1370, 1432,  
8                     1370, 1864, 1488, 1278, 1657, 1245, 1533, 1096, 1223, 1531]
```

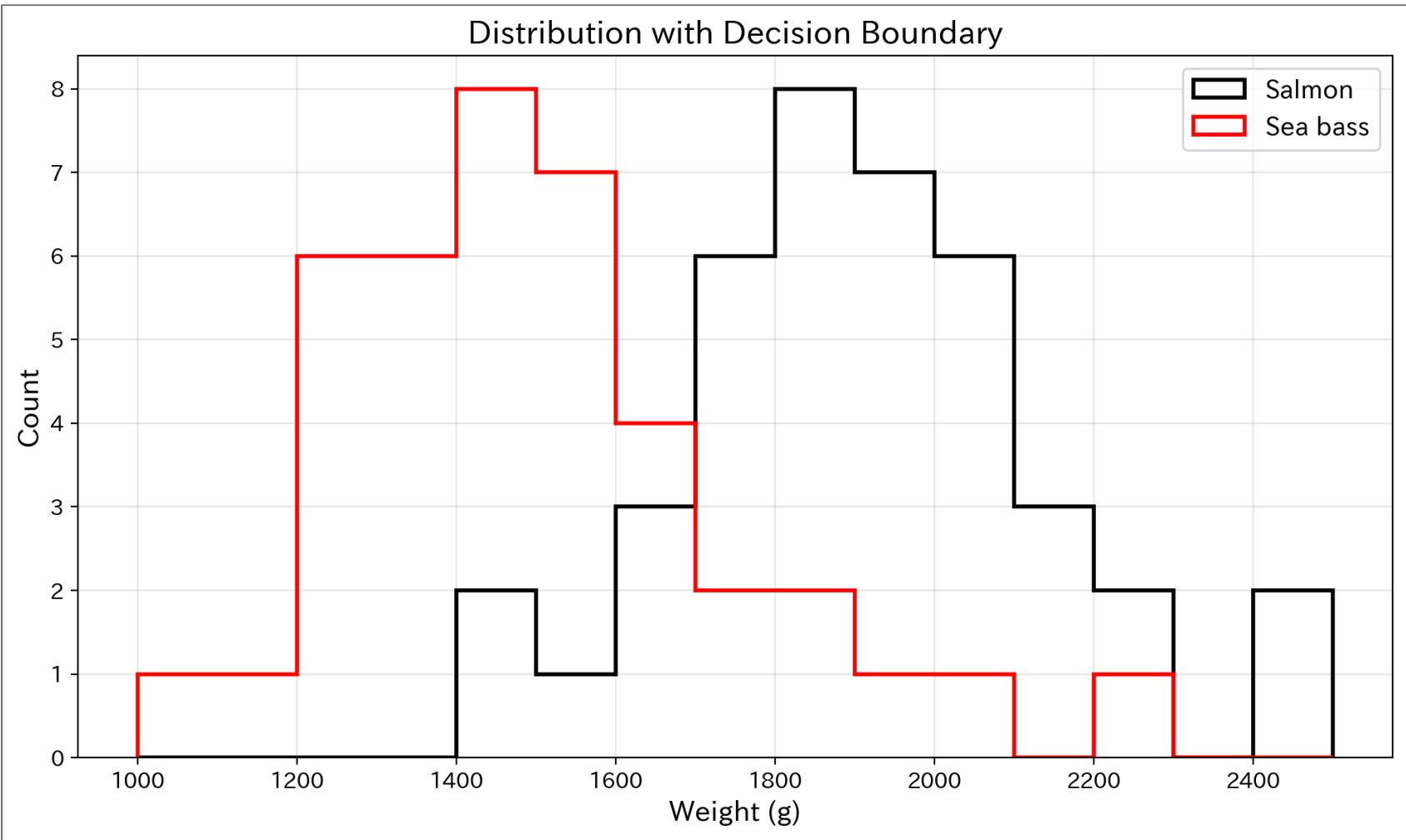
20

## 以下をレポートで提出

- 0. まず何をすべきかを考えて実行せよ。
- 1. 閾値を1800gにした場合の TP, FP, FN, TN を計算せよ
- 2. 1800gの場合の **Accuracy, Precision, Recall** を求めよ。
- 3. 1700gの場合の **Accuracy, Precision, Recall** を求めよ。
- 4.  $C_{FP} = 5000$ 円、 $C_{FN} = 1000$ 円としたとき、総コストを最小にするしきい値を求めよ

21

## 問0の答え



22